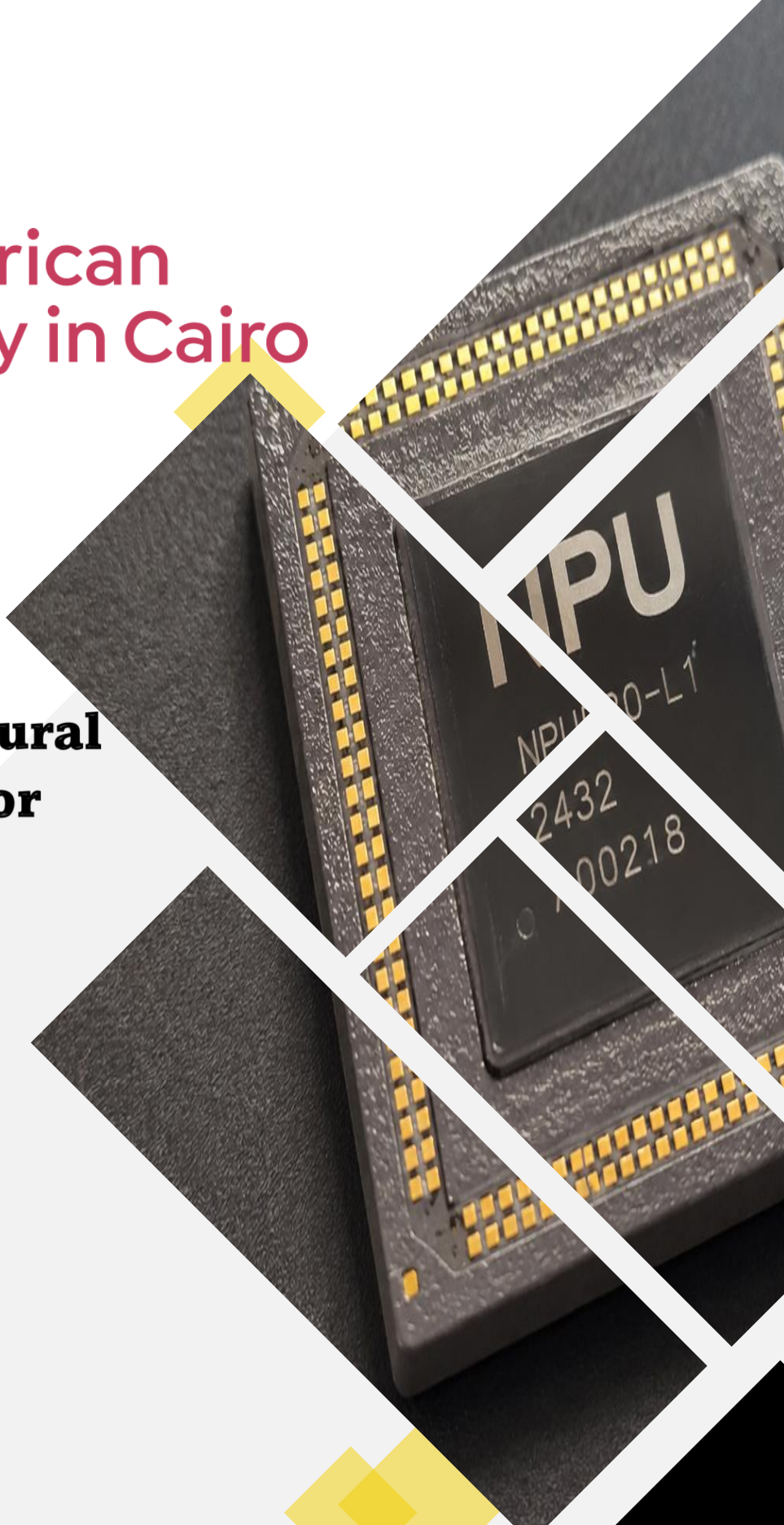
The American University in Cairo



NPU

Minimal Systolic Neural Inference Engine for Medical Edge AI

Submitted to:

Dr. Mohamed Shalan

Dr. Dina Mahmoud

Eng. Basem Hesham

Date of Submission:

May 7th

Authors:

Ammar Wahidi | Omar Eid | Mohamed Ahmed | Amr Wahidi (Software Model)

Contents

Table of Figures

Abstract

This report presents the architectural design, modern verification, and physical implementation of **nanoNPU**, a fully custom, silicon-proven Neural Processing Unit (NPU) designed for highly area-optimized medical edge inference and successfully taped out under the Silicon Sprint 2026 program at the American University in Cairo (AUC). The chip is fabricated on the SkyWater SKY130 open-source 130nm CMOS process using LibreLane and is expected to be available as a physical package in October-November 2026.

The nanoNPU implements an 8x8 systolic array executing INT8-quantized matrix multiplications at 20 MHz, capable of 128 INT8 multiply-accumulate (MAC) operations per clock cycle. It is accompanied by a fully team-designed 32-bit fixed-width Instruction Set Architecture (ISA) comprising 12 instructions, a UART-to-APB host interface, a fused post-processing pipeline (bias addition, requantization, ReLU), and a complete RTL-to-GDSII physical design flow executed using LibreLane and OpenROAD open-source EDA tools.

It is Generic NPU Works with any quantized Model. The primary validation application is a binary CNN classifier for chest X-ray pneumonia detection, trained on the Kermany et al. (Cell 2018) dataset. Full functional verification was performed using a randomized system-level testbench in QuestaSim, achieving 64/64 output word comparisons passed across all test cases. Physical signoff yielded zero DRC violations, clean LVS, and zero XOR differences across all 9 PVT corners at the worst-case slow corner (max_ss_100C_1v60).

Introduction

The deployment of deep neural network inference at the edge presents two competing constraints: computational throughput requirements demand dedicated hardware acceleration, while power budgets and die cost targets demand extreme area efficiency. General-purpose CPUs and even embedded GPUs are ill-suited to the data-parallel, arithmetic-intensive workload of quantized neural network inference at low power.

This work addresses this challenge through nanoNPU, a minimal systolic inference engine designed from first principles and physically implemented on the SkyWater SKY130 open-source 130nm PDK. Rather than adapting an existing soft-core or IP block, every component of the design was architected, coded, verified, and physically realized by the team.

The medical imaging domain was selected as the primary application context because it represents a compelling use case where edge inference has direct clinical value: a low-power chip embedded in a point-of-care device could screen chest X-rays for pneumonia without requiring cloud connectivity, reducing diagnosis latency in resource-limited settings.

Motivation

The SkyWater SKY130 PDK, combined with the open-source LibreLane / OpenROAD flow, has made it possible for academic teams to tape out real silicon without access to commercial EDA licenses or proprietary foundry processes. Silicon Sprint 2026, hosted at AUC, provided the institutional framework and multi-project chip slot that made this tapeout possible.

The design philosophy of nanoNPU prioritizes three properties: area efficiency (achieved through INT8 quantization, a minimal ISA, and a fused streaming datapath), generality (any quantized model can be programmed via the ISA), and physical correctness (all signoff checks pass cleanly at worst-case corner).

Scope of This Report

This report covers:

- System architecture and microarchitecture of all major hardware blocks
- The custom 32-bit ISA design and instruction encoding
- The Control Unit finite state machine hierarchy
- Physical design flow: synthesis, placement, routing, ECO, and signoff
- GPIO integration with the OpenFrame multi-project chip
- Functional verification methodology and results
- The software model and CNN application
- Design metrics and signoff results
- Future work roadmap

System Architecture

The implemented hardware represents a complete system-on-chip (SoC) macro environment contained within a fixed-budget Multi-Project Chip slot. The nanoNPU system consists of three hierarchical layers: the project macro wrapper that interfaces with the OpenFrame multi-project chip GPIO ring, the system top that integrates the UART-APB bridge with the NPU core, and the NPU core itself which houses the datapath, memory, and control logic.

Top-Level Hierarchy

The system architecture is partitioned hierarchically to separate external host communications, system memory arrays, and the core neural compute Datapath.

The AUC Silicon Sprint does not dedicate the entire OpenFrame user area to a single design. Instead, the chip is partitioned to host up to 12 independent student projects in a single fabrication run, arranged in a 4-row × 3-column grid.

A scan-chain-configurable MUX tree routes the chip's 38 usable GPIOs to exactly one selected project at runtime. When a project is activated, its three

orange mux macros (bottom, right, and top) connect it to the physical pads. All other projects remain electrically isolated.

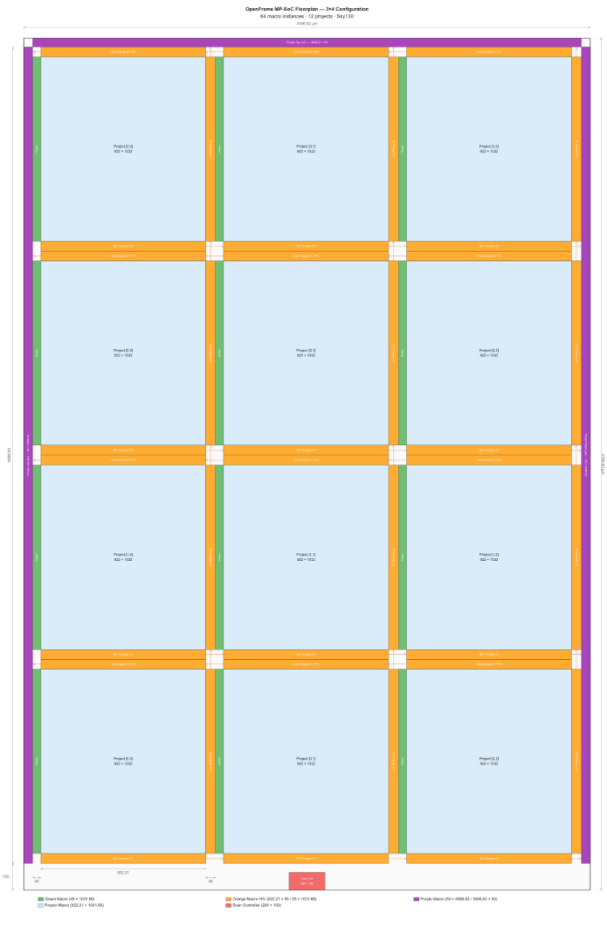


Figure 1 OpenFrame Multi-Project Floorplane

Host Interface

The host communicates with the chip exclusively through a 115200-baud UART link. The uart_apb_sys module. The UART-to-APB bridging protocol using magic-byte framing: write transactions are prefixed with the three-byte header 0xDEADA5 followed by the 4-byte address and 4-byte data, while read transactions use the header 0xDEAD5A followed by the 4-byte address. The bridge returns the 4-byte read data on the UART TX line.

The APB splitter fans the APB master bus out to three slaves: the instruction memory (IMEM) at a dedicated address range, the data SRAM (DMEM) at a second range, and the NPU control registers (start bit, done status) at a third.

Bus Interface and Memory Mapping

System control and memory load/unload transactions are orchestrated over the APB bus via the **Npu_apb_decoder**. The decoder maps an 8 KB address slice (SLOT_BITS = 13) down into individual 32-bit register configurations, an instruction window, and a data memory window:

- **CSR0 — Control Register (Offset 0x000):** Word-aligned writable configuration register.
    - Bit [0]: start_npu — Self-clearing strobe that pulses high for exactly 1 clock cycle to trigger the internal Execution Control Unit.
    - Bit [1]: load_imem — Static multiplexer control signal routing instruction memory access to the external host port.
    - Bit [2]: load_dmem — Static multiplexer control signal routing data memory access to the external host port.
    - Bit [3]: dmem_rd_host — Active-high control enabling direct asynchronous reading of data SRAM arrays by the host.
- **CSR1 — Status Register (Offset 0x004):** Read-only feedback register tracking current computation states.
    - Bit [0]: npu_done — Asserted when the execution pipeline encounters a HALT microcode state.
    - Bit [1]: done_processing — Asserted concurrently when all queued instructions are exhausted.
- **DMEM_RD_ADDR (Offset 0x008):** Latch register holding the target 8-bit word address for host-side data readbacks.
- **DMEM_RD_DATA (Offset 0x00C):** Read-only window exposing the 32-bit data word retrieved from the internal storage port.
- **Instruction Memory Window (Offsets 0x100 to 0x17C):** Direct mapped access aperture providing clear byte-enabled 32-word pipeline loading paths.

- **Data Memory Window (Offsets 0x200 to 0x5FC):** Direct access aperture exposing the 256-word dual-port data block for sequential data initialization or bulk inference data loading.

Memory Subsystem

Memory	Organization	Type	Purpose
Instruction Memory	32 x 32-bit	Single-port SRAM	Stores NPU program (ISA instructions)
Data SRAM	128 x 32-bit	Dual-port SRAM	Activations, weights, biases, output tiles
ACC Buffer	8 x 32-bit	Register file	INT32 accumulator outputs from systolic array
Bias Buffer	8 x 32-bit	Register file	Loaded bias values per output channel
Ping-Pong x2	8 x 8-bit	Register file	ACT and WGT double-buffering
Scale Register	2 x 32-bit	Register	M0 multiplier and n shift for requantization
PBias Buffer	8 x 32-bit	Register file	Post-bias-addition intermediate results
PReq Buffer	8 x 8-bit	Register file	Post-requantization INT8 results
ReLU Buffer	8 x 8-bit	Register file	Post-ReLU INT8 results

NPU Top Architecture

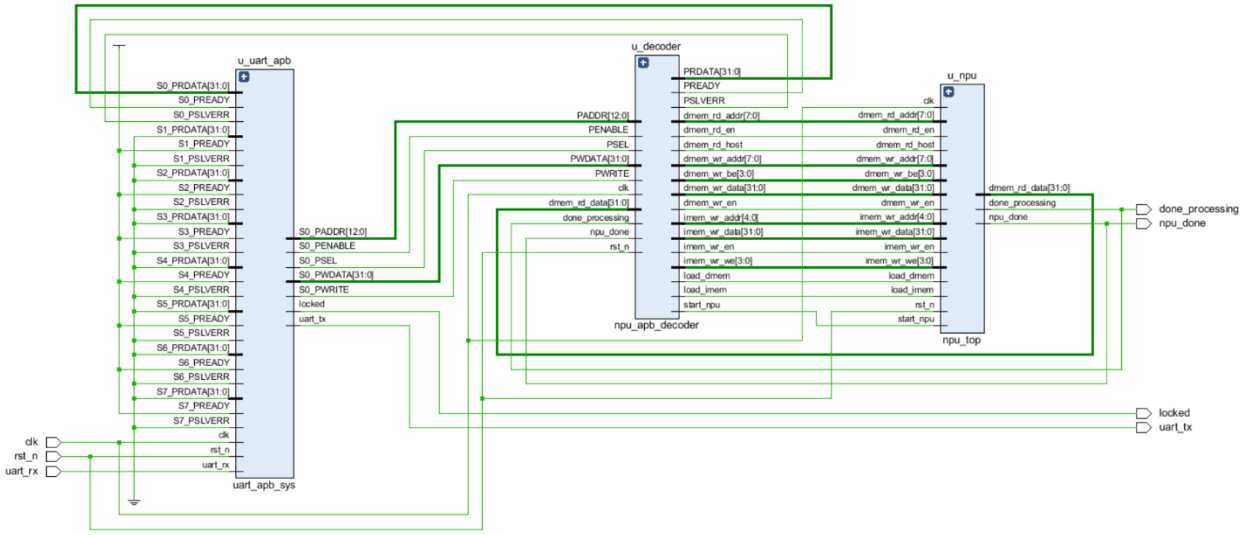


Figure 2 NPU Schame Vivado

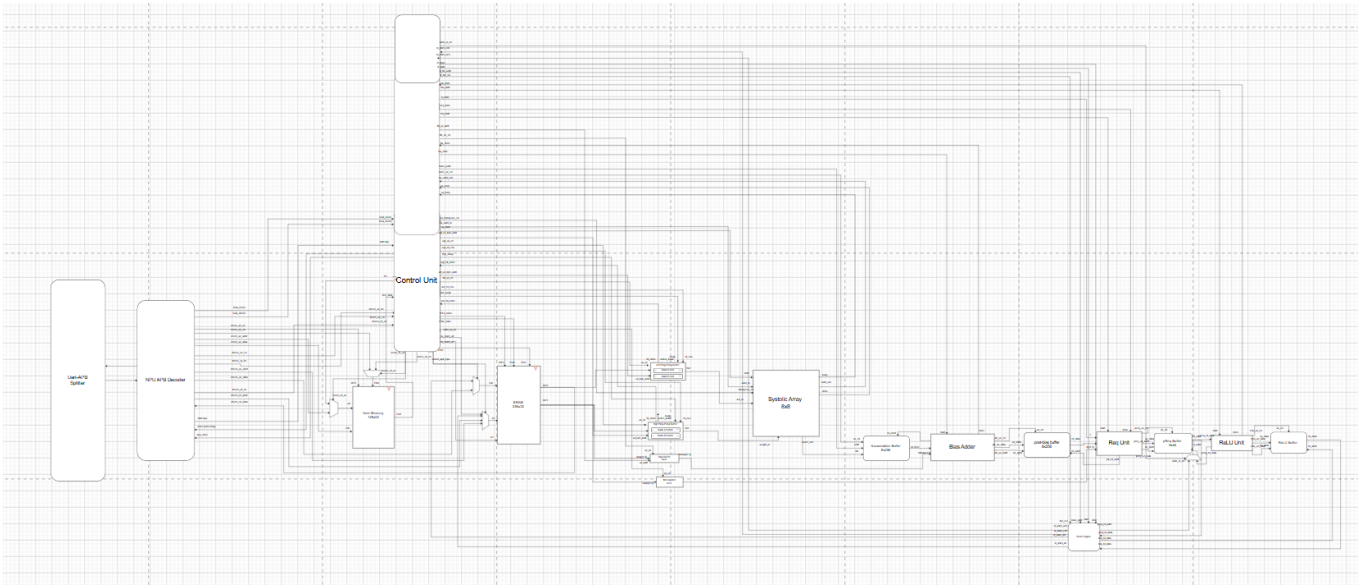


Figure 3 NPU Top Diagram

Datapath Architecture

The nanoNPU datapath implements a fused streaming pipeline in which intermediate results flow directly from one functional unit to the next through

dedicated small buffers, without returning to main SRAM between pipeline stages. This eliminates round-trip SRAM bandwidth and reduces the total on-chip buffer area to the minimum necessary to sustain the pipeline.

Systolic Array

The compute core is an 8x8 two-dimensional systolic array of Processing Elements (PEs). Each PE implements a single INT8 multiply-accumulate (MAC) operation: it multiplies an 8-bit activation input by an 8-bit weight, accumulates the 16-bit product into a 32-bit running sum, and passes both inputs to its right and bottom neighbours in the same clock cycle.

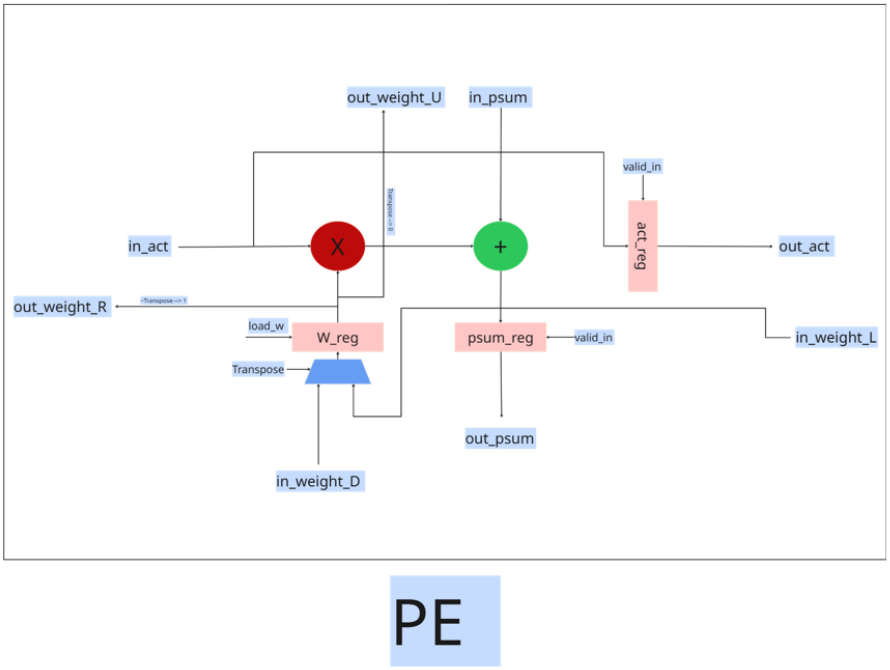


Figure 4 Processing Element (PE)

The array achieves 64 MAC operations per clock cycle. At 20 MHz, peak throughput is 1.28 GOPS (giga-operations per second) for INT8 inference.

The CONV sub-FSM first loads the weight matrix column-by-column into the array (CP_LOAD_W), then streams activations row-by-row (CP_FEED_A), stalling in CP_WAIT until the sa_done signal confirms all accumulations are complete.

SA 3x3 Datapath Example

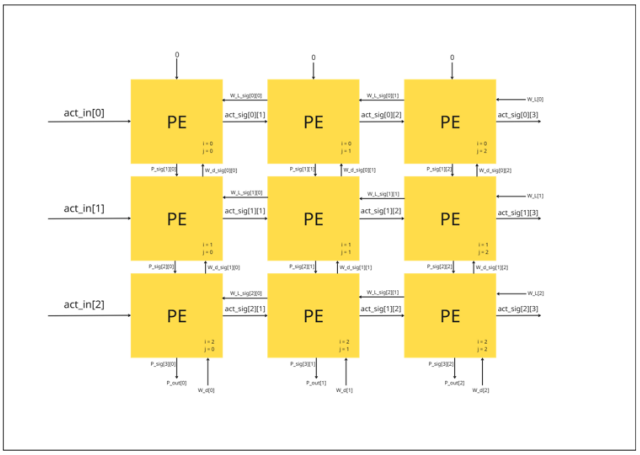


Figure 5 SA 3x3 Example

Ping-Pong Buffers

Two ping-pong buffer pairs (one for activations, one for weights) allow the Control Unit to load the next tile from SRAM into the inactive half of the buffer while the systolic array consumes the current tile from the active half. This hides SRAM access latency behind computation, improving utilization for back-to-back CONV instructions.

Post-Processing Pipeline

After the systolic array completes accumulation, the INT32 result passes through three sequential stages:

- Acc_buffer (Int32, 8 rows)
- ADD_BIAS, bias_Adder, pbias_buffer (Int32+bias)
- REQ ((acc*M0) >> n), preq_buffer (Int8)
- RELU (max(0,x), relu_buffer(Int8)
- Store

The requantization step implements the standard INT8 quantization formula: the INT32 accumulator value is multiplied by the integer multiplier M0 (loaded via LOAD_SCL), right-shifted by n bits, and clamped to the INT8 range [-128, +127] using saturating arithmetic. M0 and n together approximate the floating-

point scale factor 1/S, where S is the product of the activation and weight scale factors from the quantization-aware training process.

Instruction Set Architecture

> *The nanoNPU ISA — including all opcodes, instruction formats, field encodings, and the requantization scheme — was fully designed from scratch by the team. No existing ISA was used as a template.*

All NPU operations are encoded as 32-bit fixed-width instructions. The 6-bit opcode field in bits [31:26] identifies the operation. Two distinct instruction formats are defined depending on whether the instruction involves memory addressing or arithmetic configuration.

Instruction Formats

Load/Store Format

Bits	Field	Width	Description
[31:26]	OP CODE	6 bits	Instruction opcode
[25:22]	buf_sel	4 bits	Buffer select (ping/pong, ACT/WGT)
[21:16]	EXT_ADDR	6 bits	External (SRAM) address offset
[15:8]	TILE_ADDR_B	8 bits	Tile B start address in SRAM
[7:0]	TILE_ADDR_A	8 bits	Tile A start address in SRAM

Computation Format

Bits	Field	Width	Description
[31:26]	OP CODE	6 bits	Instruction opcode
[25:6]	RESERVED	20 bits	Reserved for future use
[5]	w_transpose	1 bit	Transpose weight matrix before loading

Bits	Field	Width	Description
[4:1]	n_scale	4 bits	Right-shift count n for requantization
[0]	BIAS_bypass	1 bit	Bypass bias addition (0 = add, 1 = skip)

Instruction Set

Instruction	OP Code	Operation
LOAD_ACT	000000	SRAM[tile] -> Activation Ping-Pong Buffer
LOAD_WGT	000001	SRAM[tile] -> Weight Ping-Pong Buffer
LOAD_BIAS	000010	SRAM[tile] -> Bias Buffer
LOAD_SCL	000011	SRAM[tile] -> Scale Register (M0, n)
CONV	000100	Act x Wgt -> AccBuffer (8x8 MAC)
ADD_BIAS	000101	AccBuffer + BiasBuffer -> PBBuffer
REQ	000110	PBBuffer x M0 >> n -> ReqBuffer (INT8, saturated)
ReLU	000111	ReqBuffer max(0,x) -> ReLUBuffer
POOL	001000	ReLUBuffer 2x2 MaxPool -> PoolBuffer (ISA defined, HW pending)
STORE	001001	LastActiveBuffer -> SRAM[tile]
LOAD_ACT_WGT	010000	SRAM -> ACT + WGT Ping-Pong simultaneously
NOP	111110	No operation, consume one clock cycle
HALT	111111	Stop execution, assert npu_done output

Control Unit Design

The Control Unit (CU.sv) is the sequencing heart of the nanoNPU. It implements a three-level FSM hierarchy: a main pipeline FSM that fetches, decodes, and dispatches instructions, a CONV sub-FSM that orchestrates the systolic array weight loading and activation streaming, and two additional sub-FSMs for REQ and ReLU row-by-row sequencing.

Main Pipeline FSM

State	Action	Next State
IDLE	Wait for start pulse from host APB register	FETCH (when start=1)
FETCH	Assert inst_rd_en, issue PC to IMEM	STALL
STALL	Latch instruction word into instr_r (1-cycle SRAM latency)	DECODE
DECODE	Decode opcode and fields, generate exec_pulse on exit	EXECUTE
EXECUTE	Drive active functional unit, hold until unit_done	NEXT (when unit_done=1)
NEXT	Increment PC, swap ping-pong if needed	FETCH / HALTED
HALTED	Assert npu_done output, hold until reset	IDLE (on reset)

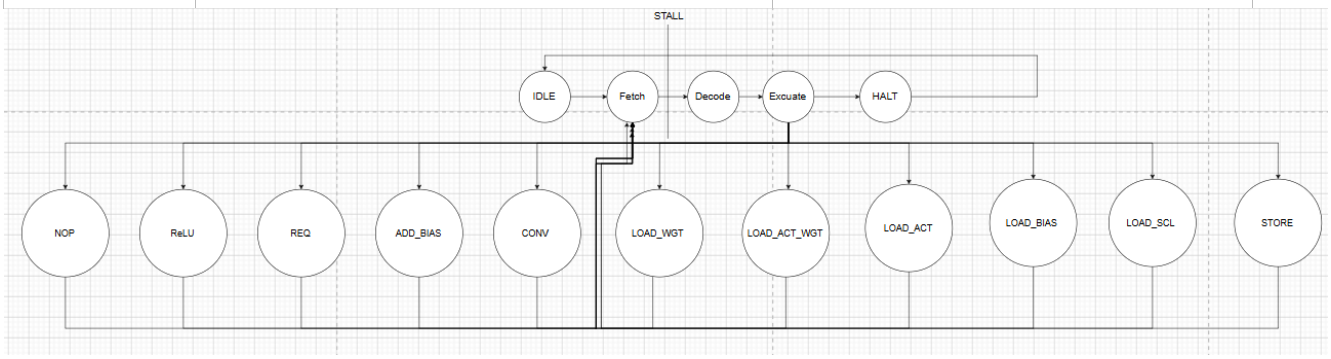


Figure 6 Main FSM

Conv Sub-FSM

When the main FSM enters EXECUTE with opcode=CONV, the SA_CU sub-controller takes over and sequences the systolic array through the following states:

State	Action
CP_IDLE	Wait for exec_pulse from main FSM
CP_START	Assert sa_start, initialize column counter to 0
CP_LOAD_W	Assert sa_valid_in, load one weight column per cycle (counts 0 to N-1)
CP_FEED_A	Assert sa_valid_in, stream one activation row per cycle (counts 0 to N-1)
CP_WAIT	Hold until sa_done asserted by systolic array, then return unit_done to main FSM

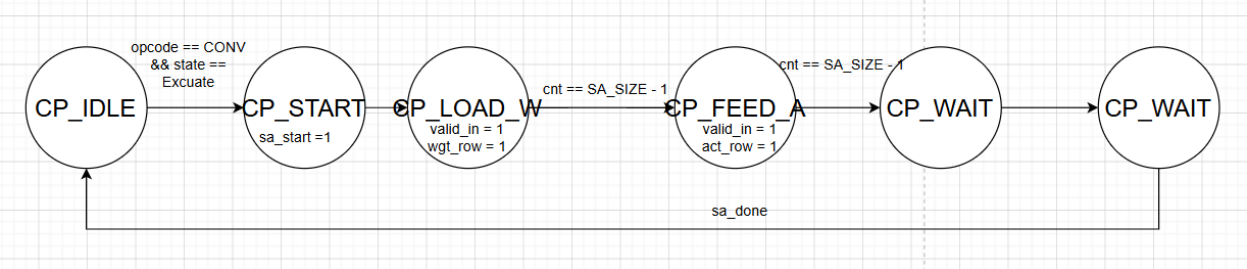


Figure 7 Conv-Sub FSM

REQ and ReLU Sub-FSMs

Both the requantization unit and the ReLU unit process results row by row. Their sub-FSMs follow the same three-state pattern:

State	Action
RQ_IDLE / RP_IDLE	Wait for exec_pulse from main FSM
RQ_PULSE / RP_PULSE	Issue start pulse for each row in sequence, count to SA_SIZE-1
RQ_WAIT / RP_WAIT	Wait for req_done / relu_done from the unit, return unit_done

GPIO Pin Map and OpenFrame Integration

The nanoNPU is wrapped inside npu_project_macro.sv, which conforms to the OpenFrame multi-project chip port convention. The wrapper connects the NPU system signals to the shared GPIO ring that OpenFrame exposes to all user projects. Only 5 of the available GPIO pins are actively driven; all remaining pins are set to safe high-impedance input mode.

Active Pin Assignment

GPIO Pin	Direction	Signal	Description
gpio_bot[0]	IN	uart_rx	Host to NPU UART receive data
gpio_bot[1]	OUT	uart_tx	NPU to host UART transmit data
gpio_bot[2]	OUT	locked	APB bus lock / busy status
gpio_bot[3]	OUT	npu_done	NPU reached HALT instruction
gpio_bot[4]	OUT	done_processing	All instructions processed
gpio_bot[14:5]	--	unused	High-Z input, no pull
gpio_rt[8:0]	--	unused	High-Z input, no pull
gpio_top[13:0]	--	unused	High-Z input, no pull

Drive Mode Encoding

Pin	oeb	dm[2:0]	Mode
BOT[0] uart_rx	1	3'b001	Input, no pull
BOT[1] uart_tx	0	3'b110	Strong push-pull output
BOT[2] locked	0	3'b110	Strong push-pull output
BOT[3] npu_done	0	3'b110	Strong push-pull output
BOT[4] done_processing	0	3'b110	Strong push-pull output

Pin	oeb	dm[2:0]	Mode
All others	1	3'b001	High-Z input, no pull

Clock and Reset

Signal	Source	Description
clk	proj_clk_out (green macro)	Gated system clock via ICG cell
reset_n	proj_reset_n_out (green macro)	Active-low reset; LOW when slot is disabled
por_n	Power-on reset	Passed through but unused directly by NPU

Pin Order Configuration

The pin_order.cfg file instructs LibreLane's I/O placer to distribute the GPIO bundles around the four die edges, matching the OpenFrame physical contract:

Die Edge	GPIO Group	Signal Count	DM Pins
West	clk, reset_n, por_n	3	0
South	gpio_bot[14:0]	15	45
East	gpio_rt[8:0]	9	27
North	gpio_top[13:0]	14	42

Physical Design

The complete RTL-to-GDSII flow was executed using LibreLane — the open-source RTL-to-GDSII orchestration framework built on OpenROAD, Yosys, Magic, and Netgen — targeting the SkyWater SKY130 HD standard cell library. The flow was run as part of the Silicon Sprint 2026 workshop at AUC.

Flow Overview

The LibreLane Classic flow was divided into multiple phases:

Phase	Key Steps	Purpose
Synthesis	Yosys SYNTH_STRATEGY=AREA 2	Logic synthesis, technology mapping
Floorplanning	DEF template, FP_CORE_UTIL=20%	Die area allocation, power grid
Placement	OpenROAD GlobalPlacement + DetailedPlacement	Standard cell placement
CTS	OpenROAD TritonCTS	Clock tree synthesis and balancing
Routing	OpenROAD GlobalRouting + DetailedRouting	Signal wire routing
Signoff Prep	FillInsertion, OpenRCX, STAPostPNR, IRDrop	Electrical and timing verification
ECO	Odb.InsertECOBuffers + re-route	Max Slew/Cap violation fix
Physical	Magic DRC, Netgen LVS, KLayout XOR	Geometric and connectivity check

Configuration Parameters

Parameter	Value	Notes
Clock Period	50 ns	20 MHz target frequency
Die Area	880 x 1031.66 um	Fixed by OpenFrame multi-project contract
Core Utilization	20%	Area-optimized; leaves routing headroom
Synthesis Strategy	AREA 2	Minimise cell count and total wire length
Default Corner	max_ss_100C_1v60	Worst-case slow, hot, low-voltage
Max Metal Layer	met4	Routing restricted to M1-M4
Antenna Repair Iters	15	Aggressive antenna violation mitigation

Parameter	Value	Notes
Post-GRT Repair	Enabled	Slew/cap fix after global routing
DIODE_ON_PORTS	in	Antenna diodes on all input ports

Parasitic Extraction and Multi-Corner STA

After detailed routing, OpenRCX extracted RC parasitics from the physical geometry. Each routed wire was modelled as a series of resistance-capacitance segments, with coupling capacitance between adjacent wires computed separately and merged below the 0.1 fF threshold. This produced three SPEF files — max, nom, and min RC corners.

Post-PnR static timing analysis (STAPostPNR) was then run across all 9 PVT corners formed by combining the three process splits with three temperature-voltage conditions:

max_ss_100C_1v60 nom_ss_100C_1v60 min_ss_100C_1v60

max_tt_025C_1v80 nom_tt_025C_1v80 min_tt_025C_1v80

max_ff_n40C_1v95 nom_ff_n40C_1v95 min_ff_n40C_1v95

ECO Buffer Insertion

The initial post-route STA revealed Max Slew and Max Cap violations at the max_ss_100C_1v60 corner. The violations were caused by overloaded driver outputs driving long high-fanout nets, resulting in slow signal transitions that exceeded the 1.5 ns slew limit and 0.051 pF capacitance limit of the sky130_fd_sc_hd cell library.

The resolution technique used was Side Load Isolation: sky130_fd_sc_hd__buf_4 cells were inserted immediately after each overloaded driver output using the INSERT_ECO_BUFFERS mechanism in config.json. The buf_4 absorbs the heavy capacitive load, leaving the original driver to charge only the small buf_4 input capacitance.

The ECO run started from the post-detailed-routing state checkpoint, re-routed only the affected nets, then re-ran the complete Signoff Prep flow to verify the fix. Post-ECO STA confirmed elimination of all Max Slew and Max Cap violations while Setup and Hold timing remained clean.

IR Drop Analysis

Static IR drop analysis was performed at max_ss_100C_1v60 — the worst-case combination of maximum wire resistance and highest current draw. Results confirmed excellent PDN health:

Network	Average Drop	Worst-Case Drop	% of Supply
VPWR (1.60V)	1.91e-05 V	1.46e-04 V	0.01%
VGND (0.00V)	1.91e-05 V	1.65e-04 V	0.01%

Layout

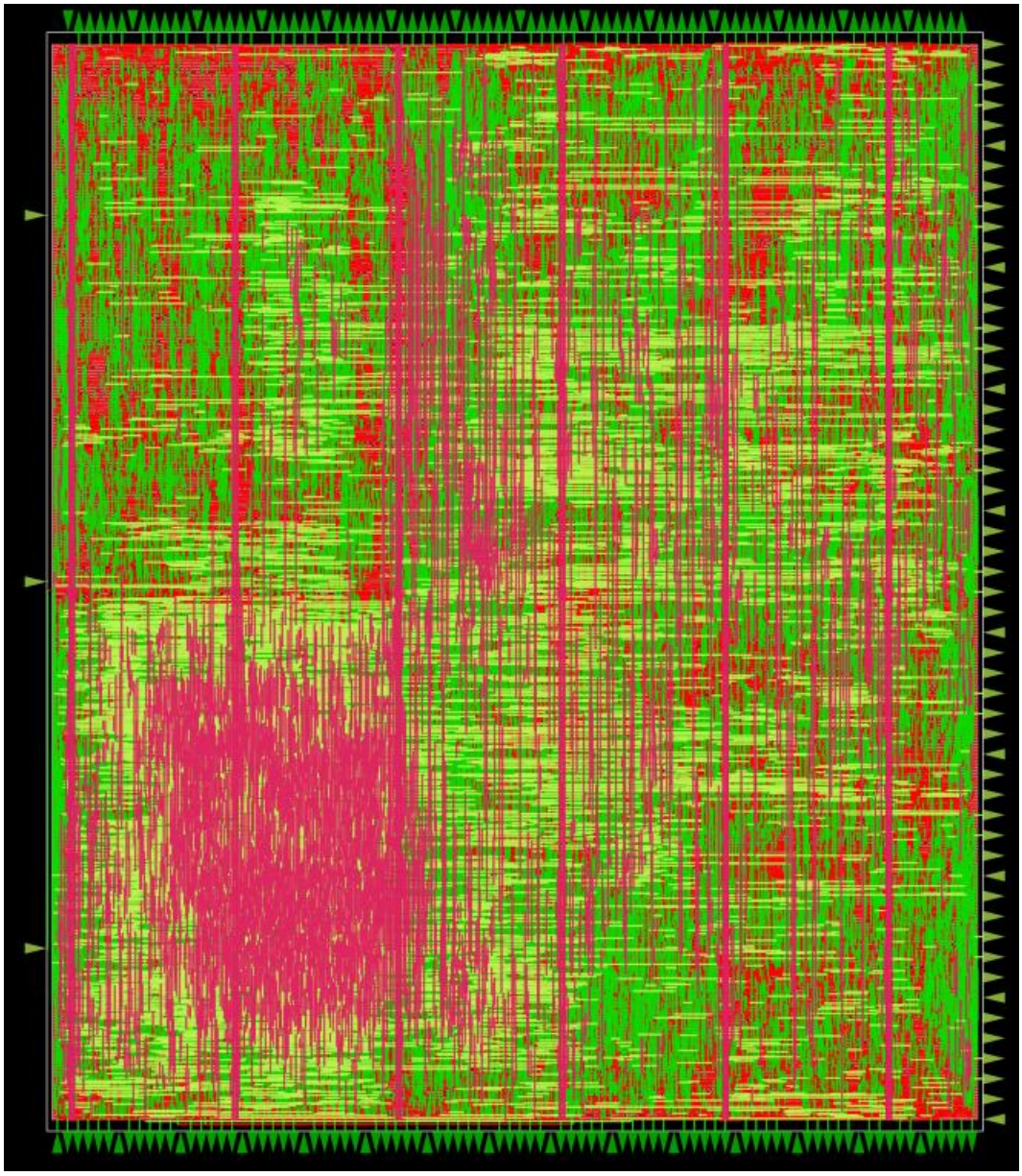


Figure 8 Layout

Placement Density

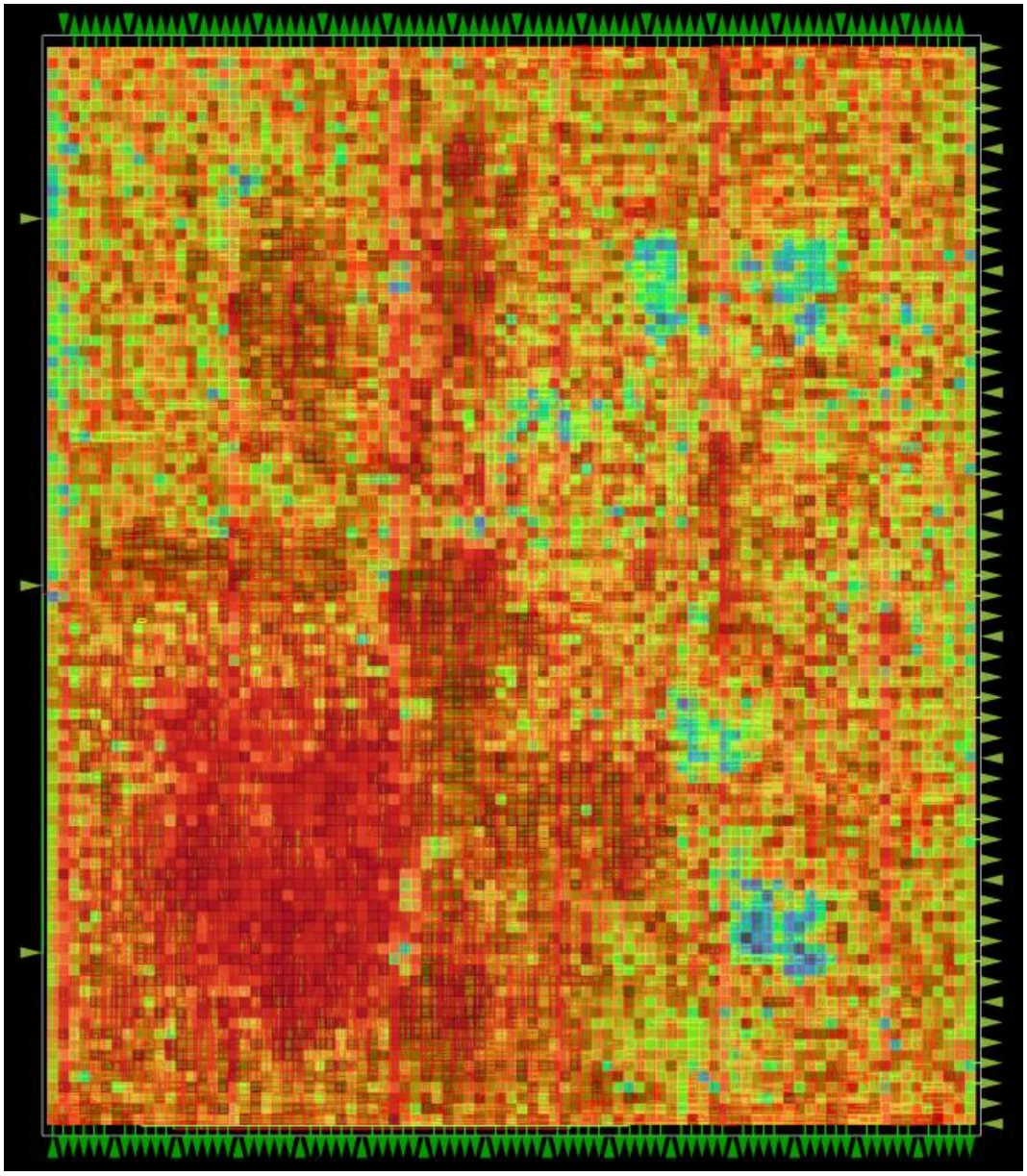


Figure 9 Placement Density

Routing Congestion

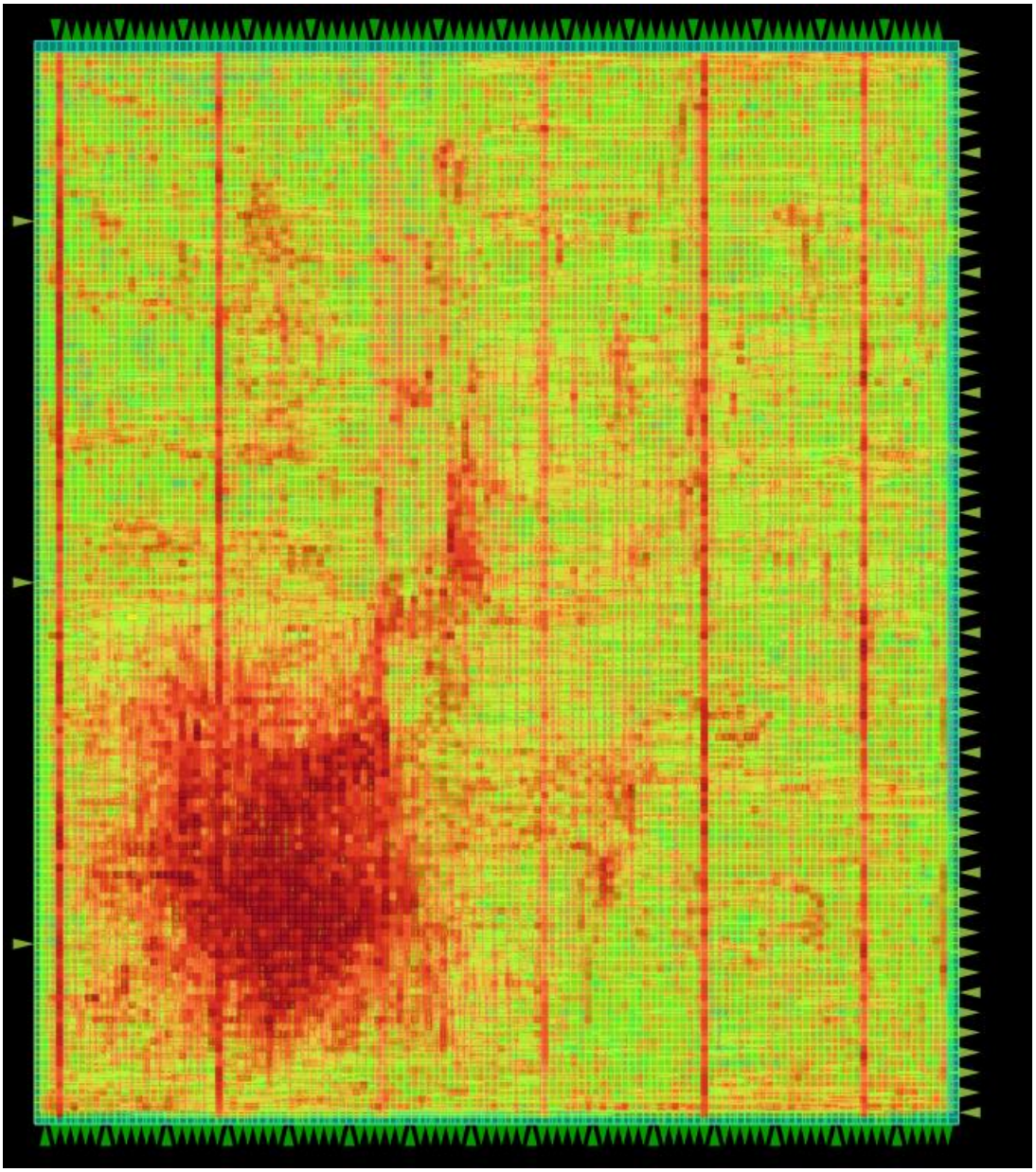


Figure 10 Routing Congestion

Design Metrics and Signoff Results

Overall Design Metrics

Metric	Value
Technology	SkyWater SKY130 130nm CMOS
Die Area	880 x 1031.66 um
Core Area	~181,090 um^2 (20% utilization)
Clock Frequency	20 MHz (50 ns period)
Peak Compute	128 INT8 MACs/cycle = 1.28 GOPS
Setup WNS (worst corner)	+0.0952 ns
Hold WNS (worst corner)	+9.0258 ns
IR Drop VPWR / VGND	0.01% / 0.01%

Physical Signoff Checklist

Check	Tool	Result	Detail
DRC	Magic + KLayout	PASS - 0 violations	Full SkyWater 130nm rule deck
LVS	Netgen	PASS - Match uniquely	Physical layout = synthesis netlist
XOR GDS	KLayout	PASS - 0 differences	Magic GDS vs KLayout GDS identical
Setup	OpenSTA (9 PVT)	PASS - 0 violations	All 9 corners clean
Hold	OpenSTA (9 PVT)	PASS - 0 violations	All 9 corners clean
Antenna	OpenROAD	PASS	Repaired via 15-iter antenna loop
IR Drop	OpenROAD	PASS - 0.01%	Well within 2% budget

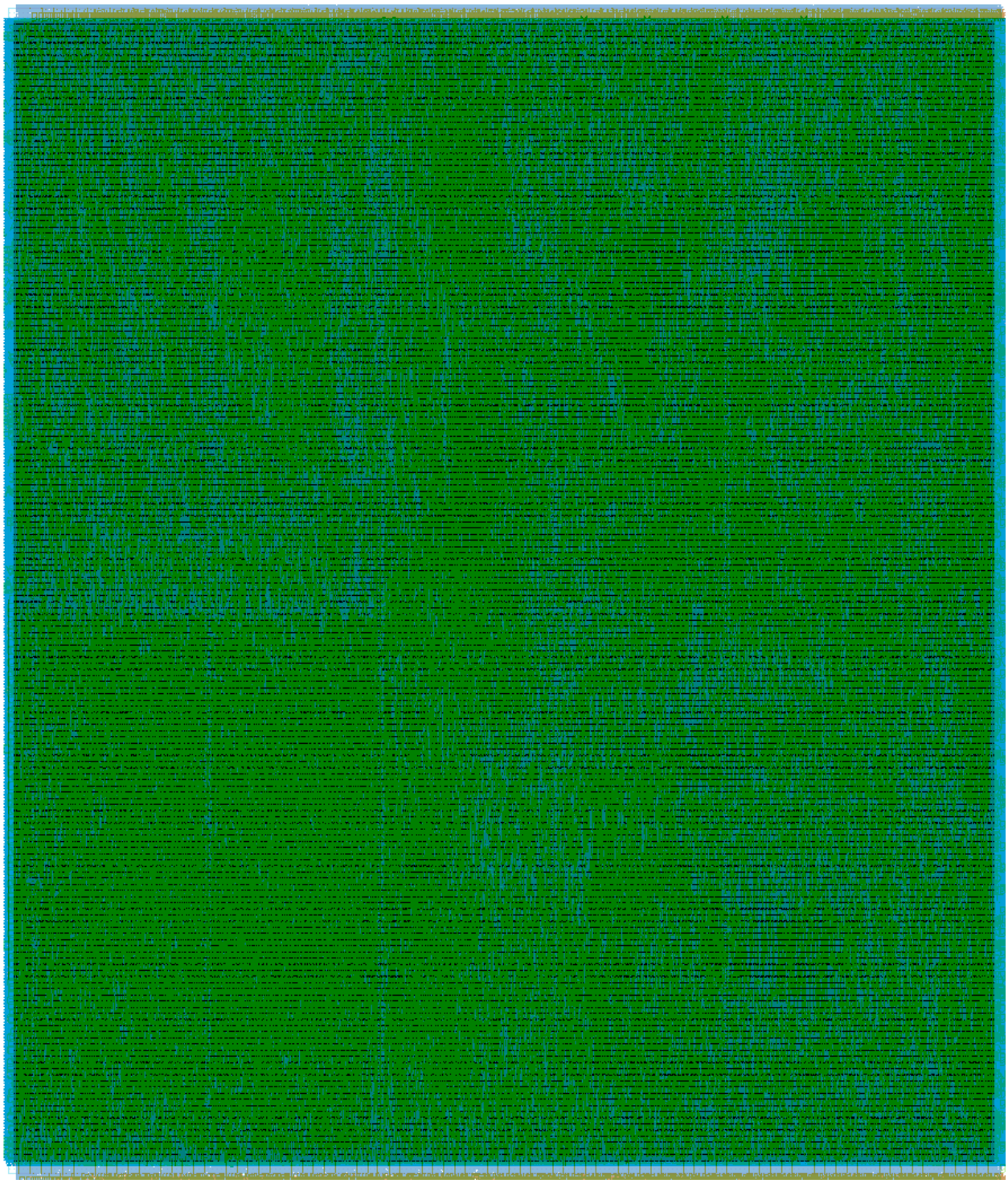


Figure 11 GDS Signoff

Functional Verification

Functional verification was performed using QuestaSim / ModelSim. The verification strategy combined a system-level randomized testbench that exercises the complete hardware stack end-to-end with a cycle-accurate golden model to automatically check every output.

Testbench Architecture

The testbench (tb_npu_system.sv) instantiates the full npu_system_top DUT and interacts with it exclusively through bit-banged UART transactions, replicating exactly how a real host processor would communicate with the chip. No internal signals are backdoor-driven; all stimulus is delivered through the serial port.

The testbench implements the following tasks:

- apb_write_uart: Serialise a 4-byte address and 4-byte data into UART frames prefixed with the 0xDEADA5 magic header
- apb_read_uart: Issue a read request using 0xDEAD5A and capture the 4-byte response
- load_sram_tile: Pack 8-bit tile data into 32-bit SRAM words and write all rows via APB
- program_imem: Encode and write the full instruction sequence into instruction memory
- golden_model: Compute expected INT8 output via matmul, bias add, requantize, optional ReLU
- check_results: Read back all 16 output words via APB and compare against golden model

Tests Cases

TC	Name	Act Range	Wgt Range	Bias Range	ReLU	Shift
TC1	Full Pipeline - Mixed Random	-20 to +20	-20 to +20	-500 to +500	Yes	2
TC2	No ReLU - Mixed Random	-50 to +50	-50 to +50	-1000 to +1000	No	4
TC3	Positive Data Only	1 to +30	1 to +30	0 to +1000	No	3
TC4	Negative Clamping - Extreme Bias	-10 to +10	-10 to +10	-8000 to +8000	Yes	0

All input data is re-randomised on each simulation run using $urandom_range, making every run a unique test vector. Each test case covers a different regime of the quantized arithmetic: TC1 is the nominal case, TC2 tests the negative value path without ReLU clamping, TC3 confirms positive-definite behaviour, and TC4 specifically stresses INT8 saturation by using extreme bias values that force many outputs to the +-127 clamp boundary.

Simulation Results

```
============================================================
  STARTING RANDOMIZED SYSTEM TESTS
============================================================

[TC1] Full Pipeline (Mixed Random Data + RELU)

[TC2] No ReLU Pipeline (Mixed Random Data)

[TC3] Positive Data Only (No ReLU)

[TC4] Negative Clamping Test (Mixed Data + RELU)

============================================================
  FINAL RESULTS: 64 PASSED, 0 FAILED
  *** SUCCESS: ALL RANDOMIZED TESTS PASSED ***
============================================================
```

Figure 12 Simulation Results

Application

The primary validation application for the nanoNPU is a binary CNN classifier that distinguishes normal chest X-ray images from pneumonia cases. This application was selected because it represents a high-value medical screening task where edge deployment has direct clinical relevance, and because the dataset is publicly available under an open license.

Figure 13 Pneumonia Cases

Dataset

Property	Detail
Source	Guangzhou Women and Children's Medical Center
Images	5,863 JPEG chest X-rays (anterior-posterior projection)
Classes	NORMAL / PNEUMONIA (binary classification)
Splits	Train / Validation / Test
Patient Age	1-5 years
License	CC BY 4.0
Citation	Kermany et al., Cell 172(5), 2018

Software Model

A bit-accurate Python software model of the full inference pipeline was developed alongside the hardware. The model implements the same fixed-point arithmetic as the hardware: INT8 activations, INT8 weights, INT32 accumulators, and requantization via the (acc * M0) >> n formula with INT8 saturation.

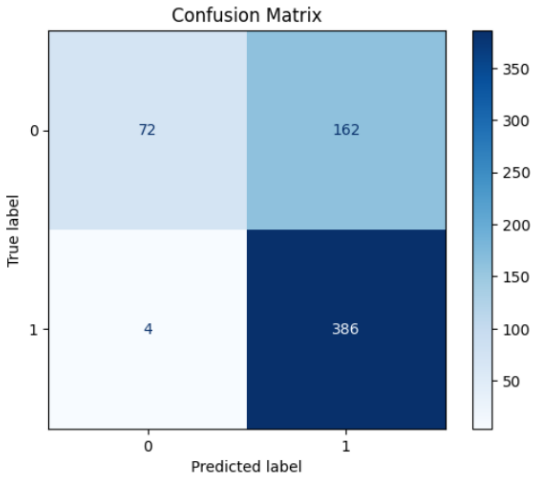


Figure 14 Confusion Matrix

The software model pipeline:

- CNN architecture definition (convolutional layers, pooling, fully connected)
- Quantization-aware training to obtain INT8-compatible weights and scale factors
- Extraction of M0 and n requantization parameters from the trained scale factors
- Weight packing into the SRAM binary layout expected by the LOAD_WGT instruction
- UART-APB instruction sequence generation for each inference pass

The software model is available as a Google Colab notebook, allowing inference to be run in a browser without any local setup. When the physical chip arrives in November 2026, the same inference will be run on silicon and results compared.

Future Work

The nanoNPU represents a working, silicon-proven baseline. The following roadmap outlines planned improvements and extensions:

Complete Pooling Unit

The POOL opcode is defined in the ISA but the hardware unit is not yet wired into the datapath. Planned: 2x2 Max Pooling (stride 2) and 2x2 Average Pooling (sum-and-shift for MobileNet global average pool).

Extended Instruction Set

Proposed additions within the existing 6-bit opcode space:

Instruction	OP Code	Operation
LOAD_ACT_WGT_BIAS	010001	Load ACT + WGT + BIAS in one pass
CONV_BIAS_REQ	010010	Fused CONV + ADD_BIAS + REQ
DEPTHWISE_CONV	010011	Depthwise separable convolution
LEAKY_RELU	010100	max(ax, x), a from scale reg
CLIP	010101	Clamp to [min, max] (ReLU6)
SIGMOID	011001	1/(1+e^-x) via INT8 LUT
SOFTMAX	011010	e^xi / sum(e^xj) over tile
SWISH	011011	x * sigmoid(x), EfficientNet
GELU	011100	x * Phi(x), Transformer blocks
HARD_SWISH	011101	x * ReLU6(x+3) / 6
LOOP	011000	Repeat next N instructions K times

Low Power Design

A clock gating cell (Clk_Gating_Cell) is already present in the RTL but not yet integrated. Planned: fine-grained ICG per functional unit, operand isolation, multi-voltage islands, and power gating with header/footer cells.

Assembler and Compiler Toolchain

Currently programs are hand-assembled. Planned toolchain: a text assembler (LOAD_ACT 0x00 0x10 -> 32-bit binary), a linker for tile address resolution, and an ONNX compiler backend that automatically tiles weight matrices and generates the full LOAD/CONV/BIAS/REQ/RELU/STORE instruction sequence.

Floating-Point NPU

A floating-point variant would eliminate the quantization requirement, enabling direct deployment of FP16/BF16 models. Key changes: replace the INT8 PE with an IEEE 754 FP16 multiply-add unit, replace the INT32 accumulator with FP32, remove the REQ unit, and widen the SRAM datapath. BF16 (8-bit exponent, 7-bit mantissa) is the recommended target for medical imaging where quantization artefacts may affect diagnostic accuracy.

Conclusion

This report has presented nanoNPU, a fully custom INT8 Neural Processing Unit designed from first principles and successfully taped out on the SkyWater SKY130 130nm open-source PDK as part of Silicon Sprint 2026 at the American University in Cairo.

The design demonstrates that a small academic team, using exclusively open-source EDA tools (LibreLane, OpenROAD, Magic, Netgen, KLayout), can carry a non-trivial digital design from RTL specification through functional verification, physical design, and full signoff to a manufacturable GDSII — without access to any commercial CAD licenses.

Key achievements: a team-designed 32-bit ISA with 12 instructions; a 64-MAC INT8 systolic array with fused post-processing pipeline; zero DRC, LVS, and XOR violations; clean Setup and Hold timing across all 9 PVT corners; IR drop below 0.05%; and 64/64 output word comparisons passed in the randomized system-level testbench.

The physical chip is expected to arrive in November 2026. Upon receipt, the validation plan is to run the chest X-ray CNN inference on silicon and confirm that the hardware output matches the bit-accurate software model.

References

- Kermany, D. et al. (2018). Identifying Medical Diagnoses and Treatable Diseases by Image-Based Deep Learning. Cell, 172(5), 1122-1131. https://doi.org/10.1016/j.cell.2018.02.010
- Chest X-Ray Images (Pneumonia) Dataset. Mendeley Data. https://data.mendeley.com/datasets/rscbjbr9sj/2. License: CC BY 4.0.
- Intel FPGA-NPU. https://github.com/intel/fpga-npu
- Ge, Y., Govindaraju, K., Jacob, S.S. (2024). Superscalar Out-of-Order NPU on FPGA. ECE5760 Final Project, Cornell University. https://people.ece.cornell.edu/land/courses/ece5760/FinalProjects/s2024/yg585_kg534_sj778
- Shalan, M. UART-APB Bridge. American University in Cairo. https://github.com/shalan
- SkyWater SKY130 PDK. https://github.com/google/skywater-pdk
- LibreLane RTL-to-GDSII Flow. https://librelane.readthedocs.io
- OpenROAD Project. https://github.com/The-OpenROAD-Project/OpenROAD
- Magic VLSI Layout Tool. http://opencircuitdesign.com/magic/
- Netgen LVS Tool. http://opencircuitdesign.com/netgen/
- KLayout. https://www.klayout.de
- OpenRCX Parasitic Extractor. https://github.com/The-OpenROAD-Project/OpenRCX

GitHub Link

https://github.com/Ammar-Wahidi/NPU